# Detecting and Attributing AI-Generated Images: A Forensic Fingerprinting Study

A Report Submitted
in Partial Fulfillment of the Requirements
for the Degree of
**Bachelor of Technology**
in
**Computer Science & Engineering**

by
**Tanish Mittal (20233288)**
**Vanshika Gautam (20233305)**
**Shashank Kanaujiya (20233259)**
**Utkarsh Raj Singh (20233298)**
**Ashish Meena (20223058)**

to the
**COMPUTER SCIENCE AND ENGINEERING DEPARTMENT**
MOTILAL NEHRU NATIONAL INSTITUTE OF TECHNOLOGY
ALLAHABAD PRAYAGRAJ
**October, 2026**

# UNDERTAKING

I declare that the work presented in this report titled "*Detecting and Attributing AI-Generated Images: A Forensic Fingerprinting Study*", submitted to the Computer Science and Engineering Department, Motilal Nehru National Institute of Technology Allahabad, Prayagraj, for the award of the ***Bachelor of Technology*** degree in ***Computer Science & Engineering***, is my original work. I have not plagiarized or submitted the same work for the award of any other degree. In case this undertaking is found incorrect, I accept that my degree may be unconditionally withdrawn.

October, 2026
Allahabad

(Tanish Mittal (20233288)
Vanshika Gautam
(20233305)
Shashank Kanaujiya
(20233259)
Utkarsh Raj Singh
(20233298)
Ashish Meena (20223058))

# CERTIFICATE

Certified that the work contained in the report titled "*Detecting and Attributing AI-Generated Images: A Forensic Fingerprinting Study*", by *Tanish Mittal (20233288)*
*Vanshika Gautam (20233305)*
*Shashank Kanaujiya (20233259)*
*Utkarsh Raj Singh (20233298)*
*Ashish Meena (20223058)*, has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

---

(Dr. Anuja Dixit)
Computer Science and Engineering Dept.
M.N.N.I.T, Allahabad

October, 2026

# Abstract

AI image generators now produce photorealistic images, which raises a forensic question: was an image captured by a camera, or produced by a generative model, and if so, by which one? This project treats AI-generated image detection as a *source identification* problem. The generator plays the role that a camera plays in classical media forensics, and it is expected to leave its own statistical fingerprint in the pixels. We evaluate detectors in four stages: real-versus-AI detection, generalization to generators unseen during training, robustness to common processing (JPEG compression, resizing, cropping) and attribution to a specific generator.

A pilot study on our own images shows why careful dataset design is essential. Real and AI images could be separated with 100% accuracy using file properties alone, without any pixel analysis. After normalization, every such shortcut fell to chance level (50%).

# Acknowledgements

We thank our mentor, Dr. Anuja Dixit, for guidance throughout this project, and the Department of Computer Science and Engineering, MNNIT Allahabad, for its support.

# Contents

**Abstract** **v**

**Acknowledgements** **vi**

**1 Introduction** **1**

1.1 Motivation . . . . . 1
1.2 Problem Statement . . . . . 1
1.3 Research Questions . . . . . 2
1.4 Contributions . . . . . 2

**2 Background and Related Work** **4**

2.1 Digital Image Forensics . . . . . 4
2.2 Generator Fingerprints . . . . . 4
2.3 Learned Detectors . . . . . 5
2.4 Datasets and Dataset Bias . . . . . 5

**3 Proposed Work** **6**

3.1 Overview . . . . . 6
3.2 Dataset . . . . . 6
3.3 Bias Control . . . . . 7
3.4 Detection Methods . . . . . 8
3.5 Training Procedure . . . . . 9
3.6 Evaluation Protocol . . . . . 10

**4 Experimental Setup and Results Analysis** **11**
4.1 Setup . . . . . 11
4.2 Pilot Study: File-Level Shortcuts (E0) . . . . . 11
4.3 Normalization (E0b) . . . . . 12
4.4 Real versus AI Detection (E1) . . . . . 13
4.5 Generalization to Unseen Generators (E2) . . . . . 14
4.6 Robustness to Image Processing (E3) . . . . . 14
4.7 Generator Attribution (E4) . . . . . 14
4.8 Effect of De-biasing (E5) . . . . . 14
4.9 Current Generators and Real-World Sharing (E6) . . . . . 15
4.10 Training on Current Generators (E7) . . . . . 15
4.11 Visualising Generator Fingerprints . . . . . 15

**5 Conclusion and Future Work** **18**

**References** **19**

# Chapter 1

# Introduction

## 1.1 Motivation

Online platforms, newsrooms and courts increasingly need to know where an image came from. Generative models such as diffusion models now produce images that are hard to tell apart from photographs by eye.

Metadata cannot answer this question reliably. In our pilot data, every real iPhone photo carried 11 EXIF tags naming the camera, while every AI image carried none. A screenshot or a messaging app strips such metadata from a real photo, and an AI image can be re-saved with any metadata at all. Active approaches, such as watermarks or signed provenance records, work only when the creator cooperates and the record survives sharing. This project therefore follows the *passive forensics* approach: it looks for traces that the image-creation process leaves in the pixel data unintentionally [8].

## 1.2 Problem Statement

Given only the pixels of an image, decide whether it was produced by a camera or by a generative model and, where possible, identify the generator. The decision should remain reliable for generators not seen during training and after common image processing.

## 1.3 Research Questions

| RQ | Question | Classical forensics analogue |
|---|---|---|
| RQ1 | Can a detector separate real photographs from AI-generated images from known generators? | Was this image captured by a camera? |
| RQ2 | Does it still work on generators never seen in training? | A camera missing from the reference database |
| RQ3 | How does performance change under JPEG compression, resizing and cropping? | Camera fingerprints weakening after processing |
| RQ4 | Can it identify *which* generator produced an image? | Camera model identification |

Table 1: Research questions mapped to classical source identification.

## 1.4 Contributions

- A de-biasing procedure for Tiny GenImage that matches real and generated images in format, resolution, colour mode and JPEG compression, including quality-factor matching and grid-aligned cropping.
- A measurement of shortcut learning: the same detector trained on the original (biased) and on the de-biased data, evaluated on the same test set.
- A comparison of three generations of detectors (a fine-tuned CNN, frozen CLIP features and the NPR forensic residual) and a simple combination of the last two, on known and unseen generators and under common image processing.
- A "then versus now" study: detectors trained on 2021–2023 generators tested on 2025–2026 generators, and, conversely, detectors trained on current generators.

- A real-world sharing test in which real and generated images are sent through WhatsApp before detection.
- Visualisations of generator fingerprints in the frequency domain and in the NPR residual.
- A demo application in which users upload an image and see the detector's prediction.

# Chapter 2

# Background and Related Work

## 2.1 Digital Image Forensics

Every step that creates or processes an image leaves statistical traces. A *trace* is any such footprint; a *fingerprint* is a trace that is consistent across all images from one source, distinctive between sources and reasonably stable under normal processing. In a camera, the lens, colour filter array, sensor, demosaicing, in-camera processing and JPEG compression each leave characteristic traces.

The best-known fingerprint is the photo-response non-uniformity (PRNU) of the camera sensor. Lukáš *et al.* [3] extract a noise residual (an image minus its denoised version), average residuals over many images from one camera to obtain a reference pattern and identify the camera of a new image by correlation.

## 2.2 Generator Fingerprints

A fully AI-generated image never passes through a lens or sensor. Instead, the generator's architecture and training can leave traces of their own. Marra *et al.* [4] showed that GANs leave model-specific fingerprints, in direct analogy with camera PRNU. This analogy is the basis of the project: detecting and attributing AI images is source identification in which the source may be a generative model.

## 2.3 Learned Detectors

Wang *et al.* [9] trained a standard image classifier (ResNet-50) on images from a single GAN and reported that, with suitable pre-processing and data augmentation, it generalized to other CNN-based generators. Ojha *et al.* [5] observed that detectors trained in this way fail to generalize to newer families of generative models. They proposed classifying images in the feature space of a pre-trained CLIP model that was never trained for real-versus-fake detection.

Tan *et al.* [6] argued that the up-sampling layers used by both GANs and diffusion models leave local correlations between neighbouring pixels. Their Neighbouring Pixel Relationships (NPR) representation subtracts from an image a copy that has been down-sampled and up-sampled again, which removes most scene content and keeps these local traces. A shallow residual network (the first two stages of a ResNet-50) is then trained from scratch on the NPR representation instead of the raw image.

## 2.4 Datasets and Dataset Bias

GenImage [10] is a benchmark of about 1.33 million real images (from ImageNet) and 1.35 million images from eight generators, with tasks for cross-generator and degraded-image classification. Grommelt *et al.* [1] showed that GenImage contains JPEG-compression and image-size biases, and that detectors trained on it learn these biases. They reported that removing the biases improved robustness to JPEG compression and raised cross-generator performance by more than 11 percentage points for ResNet-50 and Swin-T detectors.

OpenFake [2] is a recent benchmark that pairs real photographs with images from current proprietary and open-source generators. Its test split contains only generators that are absent from its training split, including GPT Image 1.5, Midjourney 7 and Flux.2, together with real images from ImageNet and DOCCI.

# Chapter 3

# Proposed Work

## 3.1 Overview

The pipeline has five stages: data collection, normalization, splitting, model training and evaluation. Every experiment saves its metrics to a shared results folder, from which the tables in this report and the demo application are produced.

## 3.2 Dataset

We use Tiny GenImage [7], a scaled-down version of GenImage with 28,000 training and 7,000 validation images. Each image is labelled as real or fake and with the model that produced it. The dataset description lists eight generators, but our inventory found no images from SD14; the seven generators present are ADM, BigGAN, GLIDE, Midjourney, SD15, VQDM and Wukong. Real images come from ImageNet. Table 2 summarises the inventory.

The two classes are balanced (14,000 real and 14,000 generated training images), but the dataset carries the same shortcuts found in our pilot study. Every real image is a JPEG and every generated image is a PNG, so the rule "PNG means fake" labels all 35,000 images correctly. Image size is also tied to the source: each generator produces a fixed resolution, so size alone separates BigGAN, Midjourney and the real images from the rest.

| Source | Train | Val. | Format | Median size (px) | Median file (KB) |
|---|---|---|---|---|---|
| Real (ImageNet) | 14,000 | 3,500 | JPEG | $500 \times 375$ | 109 |
| ADM | 2,000 | 500 | PNG | $256 \times 256$ | 117 |
| BigGAN | 2,000 | 500 | PNG | $128 \times 128$ | 29 |
| GLIDE | 2,000 | 500 | PNG | $256 \times 256$ | 75 |
| Midjourney | 2,000 | 500 | PNG | $1024 \times 1024$ | 1,211 |
| SD15 | 2,000 | 500 | PNG | $512 \times 512$ | 448 |
| VQDM | 2,000 | 500 | PNG | $256 \times 256$ | 109 |
| Wukong | 2,000 | 500 | PNG | $512 \times 512$ | 454 |

Table 2: Inventory of Tiny GenImage.

## 3.3 Bias Control

The inventory showed that file format, resolution and colour mode each reveal the source of an image. We therefore rebuilt the dataset so that real and generated images differ only in their pixel content.

1. **Filtering.** BigGAN was removed because its $128 \times 128$ images cannot be cropped to the $224 \times 224$ input size without resizing. We also removed the 280 grayscale real images and real images whose shorter side is below 224 pixels. ADM's alpha channel was dropped by converting every image to three-channel RGB. In total, 31,108 of 35,000 images remained.

2. **Cropping.** Every image was center-cropped to $224 \times 224$ pixels. We crop rather than resize, because resizing alters the fine pixel-level traces that detectors rely on. The crop offset is rounded down to a multiple of 16 pixels, so that the $8 \times 8$ JPEG block grid (and the $16 \times 16$ chroma blocks) of the real images stays aligned with the grid of the generated images.

3. **Compression matching.** All real images are JPEG files and all generated images are lossless PNG files. Following the approach of Grommelt *et al.* [1], we estimated the JPEG quality factor of each real image by matching its luminance quantization table against the standard tables scaled for qualities 1

to 100. Each generated image was then compressed once with a quality factor sampled from the distribution of real training images. After this step, every image in both classes has been JPEG-compressed exactly once.

4. **Splitting and balancing.** The dataset's training split was divided into our training (85%) and validation (15%) sets, stratified by source. The dataset's own validation split was kept as an untouched test set. Within each set, real images were sub-sampled to match the number of generated images.

All real images used standard JPEG quantization tables. Their quality factor is concentrated at 96 (12,298 of 17,500 images, 70%), with smaller groups at 75, 80, 90, 92 and 85. Table 3 gives the final split sizes and confirms that the median quality factor is the same (96) for both classes in every split.

| **Split** | **Real** | **Generated** | **Per generator** | **JPEG quality (mean / median)** | |
|---|---|---|---|---|---|
| | | | | **Real** | **Generated** |
| Train | 10,200 | 10,200 | 1,700 | 92.4 / 96 | 92.3 / 96 |
| Validation | 1,800 | 1,800 | 300 | 92.4 / 96 | 92.6 / 96 |
| Test | 3,000 | 3,000 | 500 | 94.1 / 96 | 92.4 / 96 |

Table 3: Processed dataset: six generators (ADM, GLIDE, Midjourney, SD15, VQDM, Wukong) and real ImageNet images, $224 \times 224$ pixels.

Two limitations remain. First, the generated test images were compressed with qualities drawn from the training distribution, while the real test images keep their own, slightly higher, qualities (mean 94.1 versus 92.4). Second, because generators produce different resolutions, a fixed $224 \times 224$ crop covers most of a $256 \times 256$ ADM image but only a small region of a $1024 \times 1024$ Midjourney image (Figure 1). The apparent zoom level therefore still varies with the source.

## 3.4 Detection Methods

- **M1, trained CNN:** a ResNet-50 pre-trained on ImageNet and fine-tuned for real-versus-fake classification, following the approach of Wang *et al.* [9].

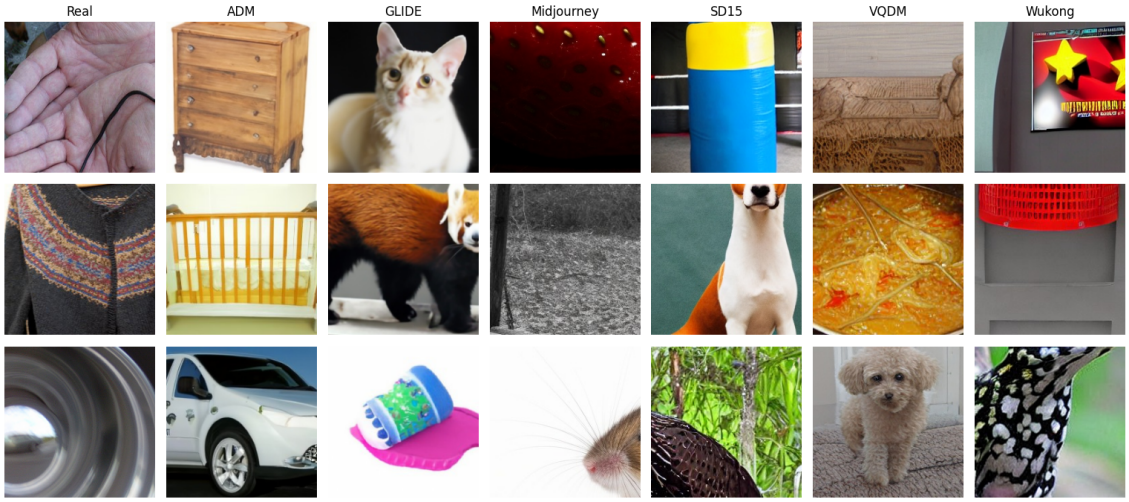



Figure 1: Processed test images, three per source.

- **M2, frozen CLIP features:** image features from a pre-trained CLIP model with a linear classifier on top, following Ojha *et al.* [5].
- **M3, NPR:** the neighbouring pixel relationship representation of Tan *et al.* [6], fed to a shallow ResNet (the first two stages of ResNet-50) trained from scratch, as in the authors' released code.
- **M4, combined detector:** the output scores of M2 and M3 combined, so that a semantic detector and a pixel-trace detector contribute to one decision.

The three published methods represent three stages of the field: a detector trained on raw pixels (2020), a detector built on a large pre-trained vision model (2023), and a detector built on a forensic residual (2024).

## 3.5 Training Procedure

All trained detectors share one training procedure. Images are scaled to $[0, 1]$, normalised with the ImageNet channel means and standard deviations, and flipped horizontally at random during training. The network outputs a single score per image, which is passed through a sigmoid to give the probability that the image is

generated. Training minimises the binary cross-entropy loss with the AdamW optimiser (learning rate $10^{-4}$, weight decay $10^{-4}$), batches of 64 images, mixed-precision arithmetic and a fixed random seed (42), for 5 epochs. After every epoch the model is evaluated on the validation set, and the checkpoint with the lowest validation loss is kept. The test set is used once, with that checkpoint.

## 3.6 Evaluation Protocol

We report four measures on the test set:

- **Accuracy:** the fraction of images classified correctly when an image is called generated if its probability is at least 0.5.
- **ROC-AUC:** the probability that a randomly chosen generated image receives a higher score than a randomly chosen real image. It does not depend on the 0.5 threshold, so it separates how well a detector ranks images from where its threshold lies.
- **Average precision (AP):** the area under the precision–recall curve, commonly reported in the detection literature.
- **Per-source accuracy:** accuracy computed separately for real images and for each generator, so that a weak generator is not hidden by the overall average.

Misclassified images are inspected in the error analysis.

# Chapter 4

# Experimental Setup and Results Analysis

## 4.1 Setup

All experiments run on Google Colab with an NVIDIA T4 GPU, using Python, PyTorch, Pillow, NumPy and pandas. Data and results are stored on Google Drive.

## 4.2 Pilot Study: File-Level Shortcuts (E0)

We collected five real photographs (Apple iPhone 17, HEIC) and five images generated with ChatGPT (PNG). Table 4 shows that the two classes differ on every file-level property.

| **Property** | **Real (5)** | **AI-generated (5)** |
|---|---|---|
| Format | HEIF | PNG |
| Resolution (W×H) | $4284 \times 5712$ | $512 \times 512$ or $768 \times 512$ |
| File size | 2,404–5,631 KB | 329–524 KB |
| EXIF tags | 11 | 0 |
| Embedded colour profile | Yes | No |

Table 4: File properties of the pilot dataset.

A *shortcut rule* labels an image as AI-generated from one file property, without reading any pixel values. Accuracy is the fraction of images labelled correctly.

## 4.3 Normalization (E0b)

Each image was rotated upright using its EXIF orientation, converted to RGB, center-cropped to $512 \times 512$ pixels, rebuilt from its raw pixel array and saved as PNG. A first attempt without the rebuilding step leaked the iPhone colour profile into all real images, which created a new shortcut. Table 5 compares the rules before and after normalization.

| **Shortcut rule (predicts "AI" if ...)** | **Before (E0)** | **After (E0b)** |
|---|---|---|
| File format is PNG | 100% | 50% |
| Width under 1,000 pixels | 100% | 50% |
| File smaller than 1 MB | 100% | 50% |
| No EXIF tags | 100% | 50% |
| No embedded information | 100% | 50% |

Table 5: Accuracy of file-level shortcut rules on the pilot set.

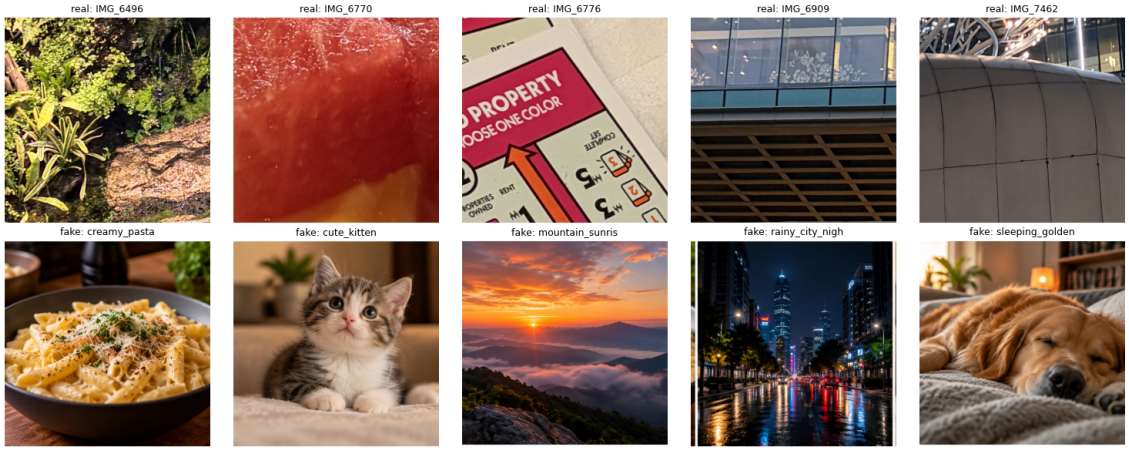



Figure 2: Pilot images after normalization. Top: real; bottom: AI-generated.

Two biases remain that file cleaning cannot remove. Real crops show close-up

textures, while AI images show centred, well-lit whole scenes (Figure 2). Real pixels also passed through in-camera compression, while AI pixels did not.

## 4.4 Real versus AI Detection (E1)

E1 trains each detector on all six generators and tests it on the held-out test set of the same generators.

### M1: fine-tuned ResNet-50

On the test set, M1 reached an accuracy of 96.5%, a ROC-AUC of 0.994 and an AP of 0.994 (Table 6). Training took about 71 seconds per epoch on a T4 GPU.

| **Method** | **Accuracy** | **ROC-AUC** | **AP** | **Test loss** |
|---|---|---|---|---|
| M1 (ResNet-50) | 0.965 | 0.994 | 0.994 | 0.104 |

Table 6: E1: test results on the six training generators.

Accuracy differs between sources. Real images were classified correctly 97.2% of the time. Among the generators, GLIDE (99.6%) and SD15 (98.2%) were the easiest to detect, while ADM (91.6%) and Midjourney (93.6%) were the hardest (Table 7).

| | **Real** | **ADM** | **GLIDE** | **MJ** | **SD15** | **VQDM** | **Wukong** |
|---|---|---|---|---|---|---|---|
| M1 | 0.972 | 0.916 | 0.996 | 0.936 | 0.982 | 0.970 | 0.954 |

Table 7: E1: per-source test accuracy (for Real, images correctly called real; for generators, images correctly called generated).

Figure 3 shows that the training loss fell from 0.25 to 0.03 over five epochs while the validation loss stayed between 0.12 and 0.15. The network continued to fit the training images more closely without improving on unseen ones, so most of what it learned in later epochs did not generalise. Keeping the checkpoint with the lowest validation loss (epoch 4) guards against this.

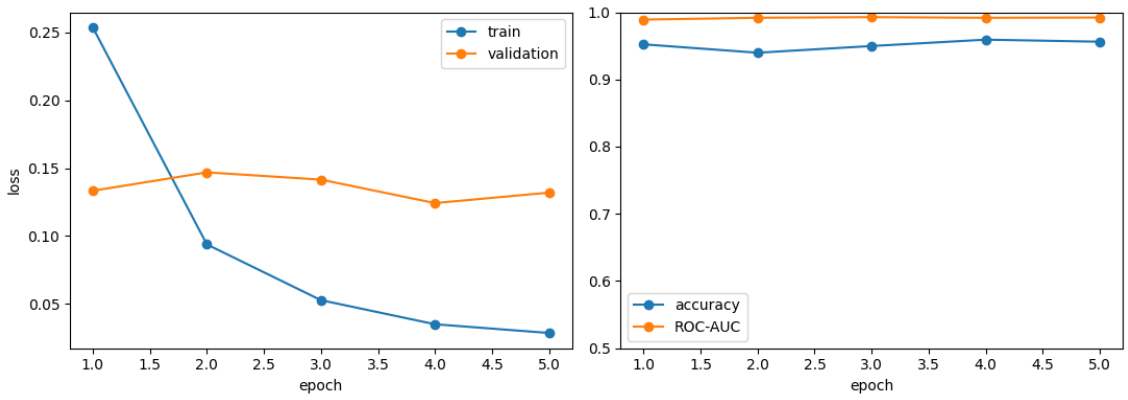



Figure 3: M1 training curves. Training loss keeps falling while validation loss stays near 0.13, a sign of over-fitting after the first epoch.

## 4.5 Generalization to Unseen Generators (E2)

*To be completed.*

## 4.6 Robustness to Image Processing (E3)

*To be completed.*

## 4.7 Generator Attribution (E4)

*To be completed.*

## 4.8 Effect of De-biasing (E5)

*To be completed.*

## 4.9 Current Generators and Real-World Sharing (E6)

*To be completed.*

## 4.10 Training on Current Generators (E7)

*To be completed.*

## 4.11 Visualising Generator Fingerprints

A single image does not reveal a generator's traces because the scene dominates. Following the logic of camera fingerprinting, we removed the scene and averaged over many images, so that content cancels out and only consistent traces remain.

### Frequency spectra of the noise residual

For each test image we converted it to greyscale, subtracted a copy blurred with a Gaussian filter ($\sigma = 1$) to obtain the noise residual, and computed the magnitude of its two-dimensional Fourier transform. Figure 4 shows the logarithm of the average spectrum over 500 images per source (top) and its difference from the real-image average (bottom). The horizontal and vertical lines through the centre appear in every source, including real images; they come from the image borders rather than from any generator.

Three observations stand out:

- **Generated images carry less high-frequency energy than real photographs.** The difference maps are mostly blue towards the edges, so generated images are smoother at the finest scale.
- **GLIDE has the most distinctive spectrum.** Its energy is confined to a diamond-shaped central region, and the corners are almost empty. This

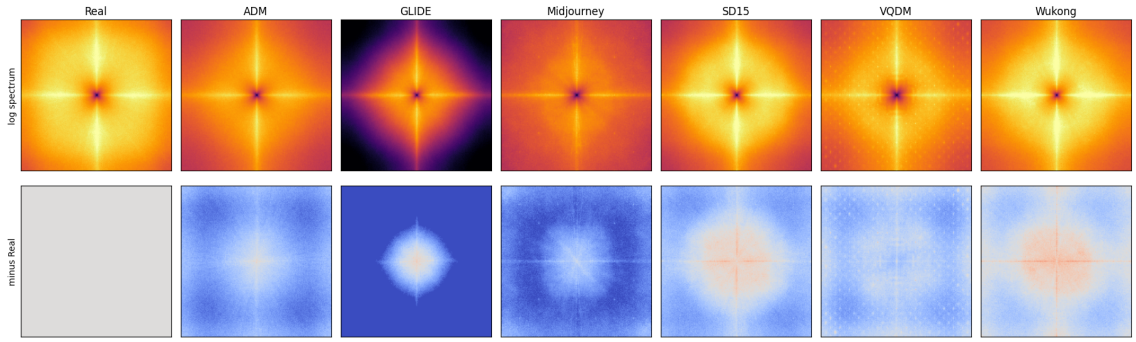



Figure 4: Average log spectrum of the noise residual per source (top) and difference from real images (bottom; blue = weaker than real, red = stronger).

pattern is consistent with an image that was produced at a lower resolution and enlarged. GLIDE was also the easiest generator for M1 (99.6% accuracy).

- **VQDM and, more faintly, Midjourney show a regular grid of bright points**, the periodic peaks expected from repeated up-sampling. SD15 and Wukong have spectra close to each other and close to real images.

## NPR residuals

Figure 5 shows a $64 \times 64$ region of one test image per source and the magnitude of its NPR residual. In the real image the residual follows the edges of the scene. In the SD15 example, a nearly uniform blue region still produces a dense, regular residual pattern, which is a trace of the generation process rather than of the content. Table 8 lists the mean absolute NPR value over 500 images per source. Every generator has a lower value than real images, meaning neighbouring pixels are more alike in generated images. GLIDE has the lowest value (3.6 against 9.6 for real images), while SD15 and Wukong are closest to real images.

| | **Real** | **ADM** | **GLIDE** | **MJ** | **SD15** | **VQDM** | **Wukong** |
|---|---|---|---|---|---|---|---|
| Mean \|NPR\| | 9.63 | 5.79 | 3.60 | 5.05 | 8.55 | 7.15 | 9.03 |

Table 8: Mean absolute NPR residual per source (pixel units, 500 images each).

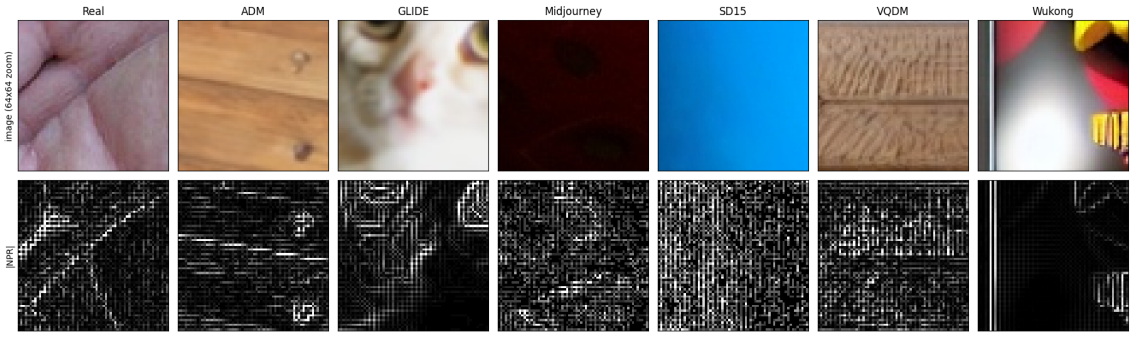



Figure 5: Image regions ($64 \times 64$, top) and the magnitude of their NPR residual (bottom).

# Chapter 5

# Conclusion and Future Work

*To be completed.*

# References


[1] Grommelt, P., Weiss, L., Pfreundt, F.-J., and Keuper, J. Fake or JPEG? revealing common biases in generated image detection datasets. In *European Conference on Computer Vision (ECCV) Workshops* (2024). arXiv:2403.17608.

[2] Livernoche, V., Arodi, A., Musulan, A., Yang, Z., Salvail, A., Caron, G. M., Godbout, J.-F., and Rabbany, R. OpenFake: An open dataset and platform toward real-world deepfake detection. arXiv:2509.09495, 2025. Dataset: `https://huggingface.co/datasets/ComplexDataLab/OpenFake` (v2).

[3] Lukáš, J., Fridrich, J., and Goljan, M. Digital camera identification from sensor pattern noise. *IEEE Transactions on Information Forensics and Security 1*, 2 (2006).

[4] Marra, F., Gragnaniello, D., Verdoliva, L., and Poggi, G. Do GANs leave artificial fingerprints? In *IEEE Conference on Multimedia Information Processing and Retrieval (MIPR)* (2019). arXiv:1812.11842.

[5] Ojha, U., Li, Y., and Lee, Y. J. Towards universal fake image detectors that generalize across generative models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2023).

[6] Tan, C., Liu, H., Zhao, Y., Wei, S., Gu, G., Liu, P., and Wei, Y. Rethinking the up-sampling operations in CNN-based generative network for

generalizable deepfake detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2024).

[7] TheKernel01. Tiny GenImage dataset. Hugging Face Datasets, `https://huggingface.co/datasets/TheKernel01/Tiny-GenImage`. Subset of GenImage; licence CC-BY-NC-SA-4.0; accessed 17 October 2026.

[8] Verdoliva, L. Media forensics and deepfakes: An overview. *IEEE Journal of Selected Topics in Signal Processing 14*, 5 (2020), 910–932.

[9] Wang, S.-Y., Wang, O., Zhang, R., Owens, A., and Efros, A. A. CNN-generated images are surprisingly easy to spot... for now. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2020).

[10] Zhu, M., Chen, H., Yan, Q., Huang, X., Lin, G., Li, W., Tu, Z., Hu, H., Hu, J., and Wang, Y. GenImage: A million-scale benchmark for detecting AI-generated image. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track* (2023). arXiv:2306.08571.